ACIT 4850 – Lab 9 – Artifact Management

Instructor	Mike Mulder (mmulder10@bcit.ca)
Total Marks	10
Due Dates	Screenshots due by end of next class: • March 24th for Sets B and C • March 25th for Set A

Applicable Requirements

- **(New) REQ1180 –** The Enterprise Development Environment shall support an Artifact Management capability. This Artifact Management capability will be Sonatype Nexus 3.
- **(New) REQ1190 –** The Enterprise Development Environment Continuous Integration capability shall retrieve and publish artifacts from an Artifact Repository.

Group Work

You will be working on the same Azure cloud environment as the previous lab, shared with your Lab partner. This lab will be done together with your partner.

You will need the following four applications running for this lab:

- Apache
- GitLab
- Jenkins
- Nexus

<u>Part 0 – Update your Java Agent</u>

In Lab 7 we forgot to add the ssh keep alive commands to the Dockerfile for the Java Agent.

SSH onto your Jenkins VM. Stop and remove all your Jenkins Docker containers using the command:

docker compose down

Then update the Dockerfile for your Java Agent with the following:

```
# Set SSH keep-alive intervals to prevent connection loss
# Sends client keep alive ever 30 seconds and disconnects after 3
failures in a row
RUN echo "ClientAliveInterval 30" >> /etc/ssh/sshd_config
RUN echo "ClientAliveCountMax 3" >> /etc/ssh/ssd_config
```

Build your Java Agent Docker image using your docker build command.

Run all your Jenkins Docker containers using the command:

docker compose up -d

Your Java Agent shouldn't have SSH timeout issues anymore.

Part 1 – Install Nexus 3

Reuse your "SonarQube" Virtual Machine from Lab 7. Start it up on the Azure Portal and ssh into the VM.

Ensure Docker is installed on this VM. Note that it should already be installed from Lab 7. You can disable your SonarQube installation and remove the Quality Gate stage from your pipeline for this lab Note: You will also have to tweak the Build stage so it doesn't try to send the analysis to SonarQube, i.e., remove withSonarQubeEnv wrapping the sh command and the sonar:sonar from the mvn command).

Run the Nexus 3 Docker Container:

- Make sure you are in your home directory (i.e., /home/<username>). Note: username is likely azureuser for your VM.
- sudo mkdir nexus-data
- sudo chown -R 200 nexus-data
- sudo docker run -d -p 8081:8081 --restart always --name nexus -v <*path to your nexus-data folder*>:/nexus-data -e NEXUS_CONTEXT=nexus sonatype/nexus3

  Example of <path to your nexus-data folder>: -v /home/azureuser/nexus-data:/nexus-data

  This maps the /nexus-data folder in the container to your local /home/mikem/nexus-data folder.
  The -e NEXUS_CONTEXT=nexus sets the context path to /nexus so we can add this to our reverse proxy

Nexus does not have a database. It uses the filesystem to store both artifacts and metadata. That is what the nexus-data volume is used for.

Add an inbound security rule that allows access to your Other Virtual Machine on TCP port 8081.

Add the ProxyPass and ProxyPassReverse configurations to the default-ssl.conf file of your Apache reverse proxy using the /nexus as the path and http://<Other DNS>:8081/nexus as the Nexus URL. For example:

```
AllowEncodedSlashes NoDecode

ProxyPass /nexus http://<Other DNS:8081/nexus
ProxyPassReverse /nexus http://<Other DNS>:8081/nexus
RequestHeader set X-Forwarded-Proto "https"
```

The Nexus User Interface is now available at https://<Apache DNS>/nexus

Note: It will take a minute or so for Nexus to startup after running the container.

Reference: https://hub.docker.com/r/sonatype/nexus3/
Note: There may not be enough memory to run both SonarQube AND Nexus 3 on the Other VM. If you run into a memory problem, shutdown your SonarQube and then run Nexus 3.

Part 2 – Configure Nexus 3

In your browser, go to the Nexus 3 User Interface at https://<Apache DNS>/nexus

Login as the admin user:

- Username: admin
- Password: <In the /nexus-data/admin.password file on your VM>

You will get a login Setup Wizard:

- Change your admin password to something you will remember (or store it in your password safe).
- Enable Anonymous Access

Browse the default repositories:

- Click on the Browse option on the left side of the screen
- Observe there are several Java/Maven repositories:
    - **maven-central** – This is where artifacts from the maven-central public repository will be cached
    - **maven-public** – This shows all publicly available artifacts through this Nexus 3 instance.
    - **maven-releases** – This is where release artifacts that you build from your CI pipelines will be stored
    - **maven-snapshots** – This is where CI artifacts from your CI pipelines for day-to-day builds will be stored
- Look inside the above repositories. There should be no content (i.e., they are empty).

Create a maven-group repository that is a grouping of all the above artifacts. This provides a single view to all your maven-artifacts.

- Select the Cog/Gear Wheel at the top of the page
- Select Repositories on the left
- Select Create Repository (above the list of repositories)
- Select the maven2 (group) receipe
- Set the Name to maven-group
- Add maven-snapshots, maven-central, maven-releases and maven-public as Member Repositories of this group
- Create the repository

Part 3 – Configure Jenkins for Java/Maven Artifacts

Make sure you have your GitLab and Jenkins VMs running.

Login to GitLab and navigate to your JavaMavenSample repository (from Lab 7):

Edit the pom.xml file and add the following. This should go at the end of the file, just before the </project> tag.

```
<repositories>
    <repository>
      <id>maven-group</id>
      <url>https://<Apache DNS>/nexus/repository/maven-group/</url>
    </repository>
  </repositories>
  <distributionManagement>
    <snapshotRepository>
      <id>nexus-snapshots</id>
      <url>https://<Apache DNS>/nexus/repository/maven-snapshots/</url>
    </snapshotRepository>
    <repository>
      <id>nexus-releases</id>
      <url>https://<Apache DNS>/nexus/repository/maven-releases/</url>
    </repository>
  </distributionManagement>
```

Make sure to replace <Apache DNS> with the DNS of your Apache VM.

The above tells Maven where to retrieve artifacts (maven-group) and where to publish artifacts.

Edit the ci/Jenkinsfile and add the following stage (just before the Deliver stage):

```
        stage('Deploy') {
            steps {
                sh 'mvn -B -DskipTests -s settings.xml clean deploy'
            }
        }
```

If you shutdown your SonarQube, remove the sonar:sonar from the Build stage and the Quality Gate stage altogether.

The above stage will publish the artifact (i.e., jar file) created by this CI pipeline in the Build stage to the Nexus repository.

Add a **settings.xml** file to your JavaMavenSample project to set the username and password against with to publish artifacts to Nexus. *Note: Normally you would NOT check in files with passwords in your repo.*

```
<?xml version="1.0" encoding="UTF-8"?>
<settings xmlns="http://maven.apache.org/SETTINGS/1.1.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.1.0
http://maven.apache.org/xsd/settings-1.1.0.xsd">

  <servers>
    <server>
      <id>nexus-snapshots</id>
      <username>admin</username>
      <password><Your Admin Password></password>
```

```
    </server>
    <server>
      <id>nexus-releases</id>
      <username>admin</username>
      <password>><Your Admin Password></password>
    </server>
  </servers>

  <mirrors>
    <mirror>
      <id>central</id>
      <name>central</name>
      <url>https://<Apache DNS>/nexus/repository/maven-group/</url>
      <mirrorOf>*</mirrorOf>
    </mirror>
  </mirrors>

</settings>
```

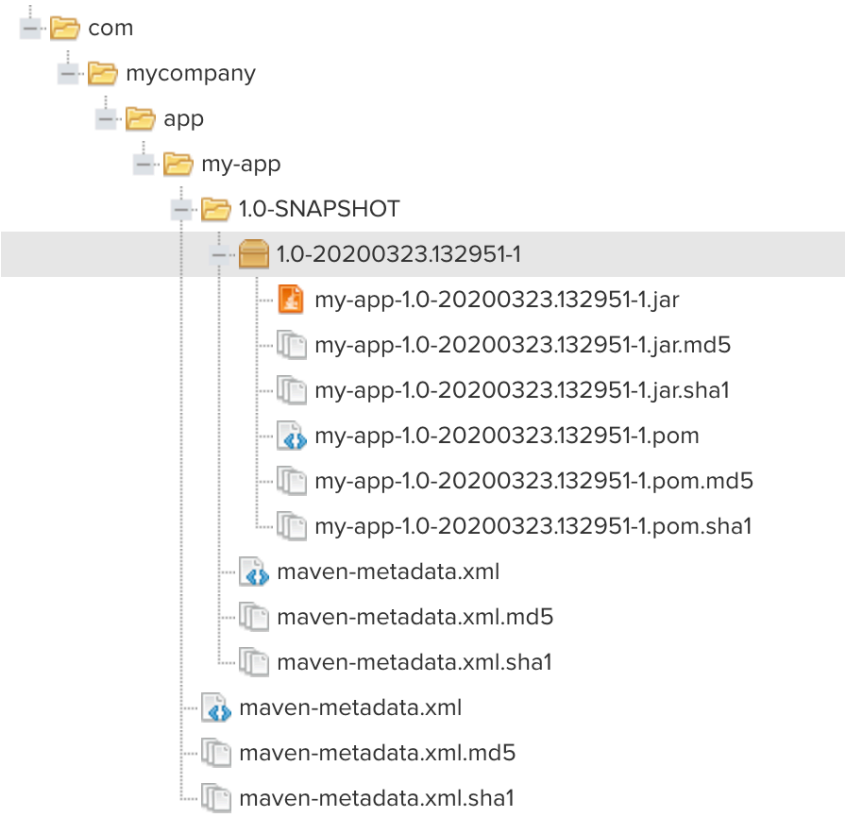
Make sure to replace <Your Admin Password> and <Apache DNS> in the settings.xml file with your values.

Run your Java CI Pipeline in Jenkins. Make sure it passes all the stages.

Part 4 – Verify the Artifacts in Nexus 3

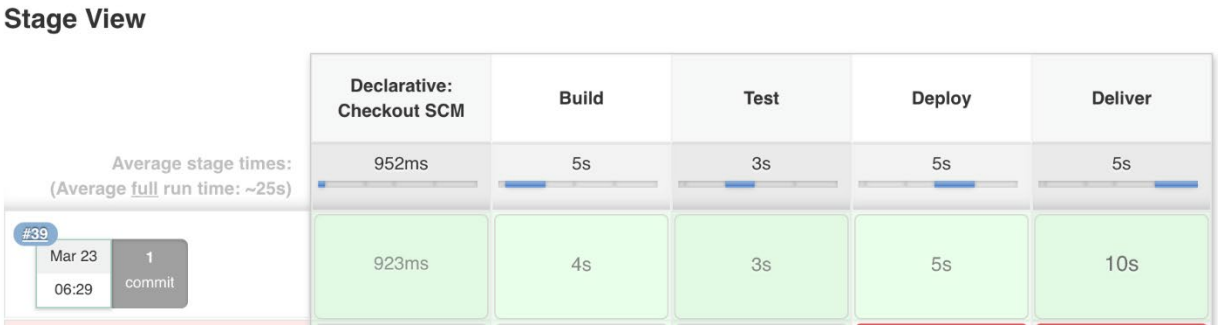
Go back to the Nexus 3 User Interface.

- Browse the repositories (select the Browse icon at the top of the page)
- Select the Browse option on the left
- Select the maven-central repository. There is now content.
    - org
- Select the maven-public repository. There is now content:
    - com
    - org
- Select the maven-releases repository. It is still empty since we did not build a release.
- Select the maven-snapshots repository. There is now content:
    - com
    - If you drill down through the folders you should find the jar file build with version 1.0-SNAPSHOT (as set in the pom.xml file)
    - It should look something like this:



Take the following screenshots:

- Nexus – The contents of the maven-snapshots repository expanded out (as above). <u>It must also include the URL of your Nexus 3 server (this should be the reverse proxied URL with your Group #).</u>
- Jenkins – The build results for your JaveMavenSample job. It will look similar to the sample below, <u>but it must also include the URL of your Jenkins server (this should be the reverse proxied URL with your Group #).</u>



<u>Make sure you shutdown any Azure resources (i.e., the VM) to conserve your credits and free tier usage.</u>

Demo, Grading and Submission

There is no demo for this lab Submit the two screenshots from Part 4 to the Lab 9 dropbox on D2L for your Group.